

Exercise 3: Inheritance

Emergency Response Personnel System

Overview

- **Take-home exercise:** to be completed **outside of class**.
- Deadline: **Sunday, 31st May 2026**, 23:59 MYT (UTC+8).
- Strategy: **Pair Programming** - two students collaborate online using VS Code Live Share.
- Submission: Submit the source code (**.cpp file**) and the **video link** via elearning before the deadline.

Pair Programming Rules

The Two Roles

 Driver	Types the code and controls the keyboard. Focus on writing clean, working code based on what you and your Navigator have discussed.
 Navigator	Reviews the code in real time, spots errors, suggests improvements, and guides the overall direction of the solution. The Navigator does not type.

Ground Rules

- Decide who will be the **host**. The host opens a Live Share session in VS Code with Read and Write permissions.
- Share the Live Share link with your partner. Use **Google Meet, Webex, or any online meeting tool** to communicate.
- Switch roles at least once. Both members must experience both the Driver and Navigator roles. You may switch after completing any task.
- Stay engaged throughout. The Navigator should speak up actively. If the Navigator is silent, the lecturer will take note.
- Record the session. See the **Video Recording** section for instructions.

Pair Programming Evidence

Since this is a take-home exercise, pair programming is assessed through **two sources of evidence**: the **video recording** of your session and the **Role Declaration comment** in your source code. There is no online log to fill in. Both sources must be consistent with each other.

Part 1: Video Recording

Record your entire pair programming session from start to finish. The video is the primary evidence that you worked together genuinely.

What the video must show:

- Both members **actively talking and discussing** throughout the session; not just one person typing in silence.

- The Driver must explain out loud what they are writing and why, for every task. For example: *"I am using inheritance here because Firefighter is a specialised type of Person, so it makes sense for it to inherit common details such as name, IC number, and phone."* This shows genuine understanding of the concept.
- The **Navigator must actively respond**; pointing out errors, asking questions, or suggesting improvements. The Navigator must not sit silently.
- At least **one role switch** must be clearly visible. When switching, the new Driver must **say out loud**: *"I am now the Driver"* or *"I am now the Navigator"*.
- Your **VS Code editor and console output** must be visible at all times. Increase the font size by pressing `Ctrl ++` in VS Code so your code is easy to read.

How to record:

- Use **Google Meet 'Record meeting'**, OBS, or any screen recorder. Use your UTM student account on Google Meet to access the recording feature.
- You may submit either a single video or multiple videos. If submitting multiple videos, place them in one Google Drive folder or provide all YouTube links together.
- Submit your recording using ONE of the following methods:
 - ✓ **Google Drive**: Set sharing to **Restricted** and add **lizawati@utm.my**.
 - ✓ **YouTube**: Upload the video as **Unlisted** (not Public or Private)
- Ensure the video remains accessible until the end of the semester.
- Submit the raw recording only. No post-editing is required.

Important - The Use of AI Tools:

You are permitted to use AI tools such as ChatGPT or GitHub Copilot **as a reference only**, in the same way you would use a textbook or lecture note. However, **submitting code that is largely generated by AI without genuine understanding is considered academic misconduct**. The purpose of this exercise is to develop your own understanding of OOP concepts. Submitting AI-generated code defeats this purpose entirely.

The video is the most reliable evidence of genuine understanding. **If your video shows you cannot explain your own code**, or if your explanation is inconsistent with what was submitted, **your submission may be called for a spot-check interview with the lecturer**. Selected pairs will be informed after submission and asked to explain their code verbally in a short session.

 **Submissions without a video, or with a video too short to confirm genuine collaboration and understanding, will receive zero marks for the pair programming component. No appeal will be entertained.**

Part 2: Role Declaration in Source Code

At the very top of your `.cpp` file, before any `#include` statements, add the following comment block. Fill in the details accurately. This declaration must be consistent with what is shown in your video.

```
// =====
// SECJ1023 - Programming Technique II
// Exercise 3: Inheritance
// Semester 2, 2025/2026
// =====
// Pair Information:
//   Member 1 (Name)      : _____
//   Member 1 (Matric No.) : _____
//   Member 2 (Name)      : _____
//   Member 2 (Matric No.) : _____
// -----
// Role Declaration:
//   Task 1 - Driver: _____ Navigator: _____
//   Task 2 - Driver: _____ Navigator: _____
//   Task 3 - Driver: _____ Navigator: _____
//   Task 4 - Driver: _____ Navigator: _____
```

```
// Task 5 - Driver: _____ Navigator: _____
// -----
// Both members confirm: the above information is accurate
// and reflects our actual pair programming session.
// =====
```

⚠ **The Role Declaration must match your video.** If the video shows Member A as Driver for Task 3 but the comment says otherwise, the video takes precedence. Inconsistencies may be raised during a spot-check.

⚠ Academic Integrity & Penalties

- You may refer to lecture notes, textbooks, and online resources. **Code that is copied or shared between pairs will be treated as plagiarism** and will result in **dismissal of the entire submission with no appeal**.
- You may use AI tools **as a reference only**; the same way you would use a textbook. **Submitting code that is substantially generated by AI without understanding is academic misconduct.** Your video and spot-check are the evidence of genuine understanding.
- Programs that cannot compile** will receive a **50% penalty on the code score**.
- Late submissions** will be penalised at **10% per 15 minutes** past the deadline.

💻 Question: Emergency Response Personnel System

Write a complete C++ program that models an **emergency response personnel system** using **inheritance**. The system manages four types of personnel: a general `Person` record, a `Firefighter`, a `Medic`, and a `ParamedicFirefighter`; a dual-certified responder who is both a firefighter and a medic at the same time.

Concepts covered in this exercise: (1) **Single inheritance:** `Firefighter` and `Medic` each inherit from `Person`. (2) **Redefined method:** each class defines its own `displayInfo()` without `virtual` or `override`. (3) **Multiple inheritance:** `ParamedicFirefighter` inherits from both `Firefighter` and `Medic`. (4) **Virtual base class:** `Person` is declared as a virtual base to prevent duplication in `ParamedicFirefighter`.

Class Hierarchy

`Person` is the shared **virtual base class**. Both `Firefighter` and `Medic` inherit from `Person` using `virtual` inheritance. `ParamedicFirefighter` then inherits from both, resulting in only **one** shared copy of `Person` inside the final object.

💡 **Tip for the Navigator:** Without `virtual` inheritance, `ParamedicFirefighter` would contain two separate copies of `Person`: one from `Firefighter` and one from `Medic`. This causes ambiguity and data duplication. The `virtual` keyword on both `Firefighter` and `Medic` ensures only one shared copy of `Person` exists inside any `ParamedicFirefighter` object.

Class Design

Declare all four classes exactly as shown below. All member functions must be declared as **prototypes** inside the class and defined **outside** using `::`.

Class 1: `Person` (virtual base class)

Category	Member	Access
Data Members	<code>string name</code>	protected
	<code>string icNumber</code>	protected
	<code>string phone</code>	protected

Member Functions	void setPerson(string, string, string)	public
	string getName() const	public
	string getIcNumber() const	public
	string getPhone() const	public
	void displayInfo() const	public

Class 2: Firefighter (inherits virtually from Person)

Category	Member	Access
Data Members	string stationCode	private
	int yearsActive	private
Member Functions	void setFirefighter(string, int)	public
	string getStationCode() const	public
	int getYearsActive() const	public
	void displayInfo() const	public
	string getExperienceLevel() const	public

Class 3: Medic (inherits virtually from Person)

Category	Member	Access
Data Members	string medicalLicenseNo	private
	string specialisation	private
Member Functions	void setMedic(string, string)	public
	string getMedicalLicenseNo() const	public
	string getSpecialisation() const	public
	void displayInfo() const	public

Class 4: ParamedicFirefighter (inherits from Firefighter and Medic)

Category	Member	Access
Data Members	string badgeID	private
	bool isDeployed	private
Member Functions	void setParamedic(string, bool)	public
	string getBadgeID() const	public
	bool getIsDeployed() const	public
	void displayInfo() const	public

Global constants: const int MAX_FF = 2; const int MAX_MEDIC = 2; const int MAX_PMF = 1; declared above all class definitions. Use these constants in all array declarations and loops.

Task 1: Class Hierarchy Setup - Virtual Base Class and Inheritance Structure

Declare all four classes in the correct order. Use the protected and private data members exactly as shown in the class design tables. Declare all member function prototypes inside each class; no function bodies inside the class definition.



Tip for the Navigator: The order of class declarations matters in C++. A class can only inherit from classes already declared above it.

Task 2: Mutator Functions with Input Validation

Write the following four mutator functions. Each must validate its inputs before storing. If any validation fails, display the exact error message shown, display "Program terminated.", and terminate the program immediately. The object must not continue to the next input or display any incomplete record.

Clarification: `main()` must validate each input immediately after it is entered, before asking for the next input. The mutator functions must still repeat the same validation before storing data.

For **Person**, `void setPerson(string n, string ic, string ph):`

- If `n` is empty, display: "Invalid name. Name cannot be empty.", then display "Program terminated." and terminate the program immediately.
- If `ic` is not exactly 12 characters or contains a non-digit character, display: "Invalid IC. IC number must be exactly 12 digits.", then display "Program terminated." and terminate the program immediately.
- If `ph` is empty, display: "Invalid phone. Phone number cannot be empty.", then display "Program terminated." and terminate the program immediately.

For **Firefighter**, `void setFirefighter(string code, int years):`

- If `code` is empty, display: "Invalid station code. Cannot be empty.", then display "Program terminated." and terminate the program immediately.
- If `years < 0` or `> 40`, display: "Invalid years active. Must be between 0 and 40.", then display "Program terminated." and terminate the program immediately.

For **Medic**, `void setMedic(string licNo, string spec):`

- If `licNo` is empty, display: "Invalid license number. Cannot be empty.", then display "Program terminated." and terminate the program immediately.
- If `spec` is empty, display: "Invalid specialisation. Cannot be empty.", then display "Program terminated." and terminate the program immediately.

For **ParamedicFirefighter**, `void setParamedic(string badge, bool deployed):`

- If `badge` is empty, display: "Invalid badge ID. Cannot be empty.", then display "Program terminated." and terminate the program immediately.
- Store `deployed` directly into `isDeployed`; no validation is needed inside `setParamedic()` because the value is already a `bool`. In `main()`, validate the deployment input immediately after it is entered and before calling `setParamedic()`: accept only 1 or 0. If the user enters any other integer, display "Invalid deployment status. Must be 1 or 0.", then display "Program terminated." and terminate the program immediately.

 **Tip for the Navigator:** Each mutator only sets its own class's data members. `setFirefighter()` only sets `stationCode` and `yearsActive`. It does not call `setPerson()`. In `main()`, you will call each mutator separately for the same object.

Task 3: Accessor Functions

Write all accessor functions for all four classes. All must be declared with `const`. Each function simply returns its corresponding data member.

For **Person**: `getName()`, `getIcNumber()`, `getPhone()`.

For **Firefighter**: `getStationCode()`, `getYearsActive()`, `getExperienceLevel()`.

***getExperienceLevel() is a derived calculation function that returns "Junior" if yearsActive is between 0 and 5, "Experienced" if yearsActive is between 6 and 15, and "Senior" if yearsActive is between 16 and 40.*

For **Medic**: `getMedicalLicenseNo()`, `getSpecialisation()`.

For **ParamedicFirefighter**: `getBadgeID()`, `getIsDeployed()`.

Task 4: Redefined displayInfo()

Each class defines its own `displayInfo()` function. This is a **redefined method**, not a virtual override. Each version uses the scope resolution operator `::` to explicitly call the base class version, avoiding any duplication of output logic. **Do not use `virtual` or `override` anywhere in this task.**

For **Person**: `::displayInfo()`:

- Display the person's name, IC number, and phone in a formatted layout. This is the foundation that all other `displayInfo()` functions build on.

For **Firefighter::displayInfo()**:

- Call `Person::displayInfo()` first using the scope resolution operator to print the shared `Person` fields.
- Then display `stationCode`, `yearsActive`, and the experience level returned by `getExperienceLevel()`.

For **Medic::displayInfo()**:

- Call `Person::displayInfo()` first using the scope resolution operator.
- Then display `medicalLicenseNo` and specialisation.

For **ParamedicFirefighter::displayInfo()**:

- Call `Person::displayInfo()` once, using `Person::displayInfo()` directly, **not** `Firefighter::displayInfo()` or `Medic::displayInfo()`. This avoids printing the `Person` fields three times.
- Then display the `Firefighter` fields: `stationCode`, `yearsActive`, and experience level. Access `stationCode` and `yearsActive` using `getStationCode()` and `getYearsActive()`, and call `getExperienceLevel()` to display the derived category.
- Then display the `Medic` fields: `medicalLicenseNo` and specialisation. Access them using `getMedicalLicenseNo()` and `getSpecialisation()`.
- Then display `badgeID` and deployment status, print "Deployed" if `isDeployed` is true, "On Standby" otherwise.

⚠ Important - Redefined vs Overridden: `displayInfo()` in this exercise is a REDEFINED method. This means: (1) do NOT use the `virtual` keyword on any `displayInfo()` declaration, (2) do NOT use the `override` keyword, and (3) when you call `displayInfo()` on an object directly (e.g. `ff[0].displayInfo()`), the compiler always calls the version that matches the declared type of the object. This is compile-time resolution, not runtime polymorphism.

Task 5: main() Function

Write a `main()` function that:

- Declares three arrays: `Firefighter ff[MAX_FF]`, `Medic med[MAX_MEDIC]`, and `ParamedicFirefighter pmf[MAX_PMF]`.
- Uses a for loop to prompt the user for each `Firefighter`'s personal details (name, IC number, phone), station code, and years active. Calls `setPerson()` and `setFirefighter()` for each.
- Uses a for loop to prompt the user for each `Medic`'s personal details, medical license number, and specialisation. Calls `setPerson()` and `setMedic()` for each.
- Uses a for loop to prompt the user for each `ParamedicFirefighter`'s personal details (name, IC number, phone), station code, years active, medical license number, specialisation, badge ID, and deployment status (1 = Deployed, 0 = On Standby). Validates each input immediately after it is entered. Calls `setPerson()`, `setFirefighter()`, `setMedic()`, and `setParamedic()` for each.
- Displays a formatted report grouped by personnel type with three sections: "=== Firefighter Records ===", "=== Medic Records ===", and "=== Paramedic-Firefighter Record ===". The `Firefighter` and `ParamedicFirefighter` outputs must include the derived experience level. Calls `displayInfo()` on each object **directly by object name** (e.g. `ff[0].displayInfo()`), not through a base class pointer.

💡 Tip for the Navigator: For the `ParamedicFirefighter` object, call all four mutators in this order: `setPerson()` first (sets name, IC, phone), then `setFirefighter()` (sets `stationCode`, `yearsActive`), then `setMedic()` (sets `medicalLicenseNo`, `specialisation`), then `setParamedic()` (sets `badgeID`, `isDeployed`). Each mutator sets different members, so none overwrite each other.

Important Notes

- Write the entire program in a **single .cpp file**.
- The .cpp file must include the Role Declaration comment at the top and the Reflection answers at the bottom.
- No constructors, destructors, or dynamic memory are required. You may include small helper functions such as `terminateProgram()`, `validateYearsActive()`, and `readInteger()` if they make immediate validation clearer.
- Do not use `virtual` or `override` keywords on any `displayInfo()` declaration. The only permitted use of `virtual` is in the inheritance declarations: `virtual public Person`.
- All member functions declared in the class must be called at least once.
- All function definitions must use the scope resolution operator `::`.
- Separate the class declaration (inside the class body) from the function definitions (outside the class).

Sample Program Output

Example 1: All records entered correctly

```
Enter Firefighter 1 name      : Ahmad Zikri
Enter Firefighter 1 IC number : 900215105432
Enter Firefighter 1 phone    : 0123456789
Enter Firefighter 1 station code: FF-KL01
Enter Firefighter 1 years active: 8
Enter Firefighter 2 name      : Rosman Nawi
Enter Firefighter 2 IC number : 851103085621
Enter Firefighter 2 phone    : 0197654321
Enter Firefighter 2 station code: FF-KL02
Enter Firefighter 2 years active: 15
Enter Medic 1 name            : Dr. Priya Devi
Enter Medic 1 IC number       : 920408025678
Enter Medic 1 phone           : 0112233445
Enter Medic 1 license number  : ML-2021-0042
Enter Medic 1 specialisation  : Emergency Medicine
Enter Medic 2 name            : Nurul Aisyah
Enter Medic 2 IC number       : 940612065123
Enter Medic 2 phone           : 0168877665
Enter Medic 2 license number  : ML-2023-0109
Enter Medic 2 specialisation  : Trauma Care
Enter Paramedic-FF name      : Hafiz Azlan
Enter Paramedic-FF IC number  : 880730085432
Enter Paramedic-FF phone     : 0134455667
Enter Paramedic-FF station code : FF-KL03
Enter Paramedic-FF years active : 12
Enter Paramedic-FF license no  : ML-2019-0007
Enter Paramedic-FF specialisation: Rescue Medicine
Enter Paramedic-FF badge ID   : PMF-001
Deployed? (1=Yes, 0=No)       : 1

=== Firefighter Records ===
Name      : Ahmad Zikri
IC Number  : 900215105432
Phone     : 0123456789
Station Code : FF-KL01
Years Active : 8
Experience  : Experienced
-----
Name      : Rosman Nawi
IC Number  : 851103085621
Phone     : 0197654321
Station Code : FF-KL02
```

```

Years Active : 15
Experience   : Experienced
-----

=== Medic Records ===
Name        : Dr. Priya Devi
IC Number   : 920408025678
Phone       : 0112233445
License No  : ML-2021-0042
Specialisation: Emergency Medicine
-----

Name        : Nurul Aisyah
IC Number   : 940612065123
Phone       : 0168877665
License No  : ML-2023-0109
Specialisation: Trauma Care
-----

=== Paramedic-Firefighter Record ===
Name        : Hafiz Azlan
IC Number   : 880730085432
Phone       : 0134455667
Station Code : FF-KL03
Years Active : 12
Experience   : Experienced
License No  : ML-2019-0007
Specialisation: Rescue Medicine
Badge ID    : PMF-001
Status      : Deployed
-----

```

Example 2: Invalid inputs

Because the program must terminate immediately after the first invalid input, each invalid-input example below represents a separate program run.

Run A: invalid name

```

Enter Firefighter 1 name      :
Invalid name. Name cannot be empty.
Program terminated.

```

Run B: invalid IC number

```

Enter Firefighter 1 name      : Ahmad Zikri
Enter Firefighter 1 IC number : 9002AB105432
Invalid IC. IC number must be exactly 12 digits.
Program terminated.

```

Run C: invalid phone

```

Enter Firefighter 1 name      : Ahmad Zikri
Enter Firefighter 1 IC number : 900215105432
Enter Firefighter 1 phone     :
Invalid phone. Phone number cannot be empty.
Program terminated.

```

Run D: invalid station code

```

Enter Firefighter 1 name      : Ahmad Zikri
Enter Firefighter 1 IC number : 900215105432
Enter Firefighter 1 phone     : 0123456789
Enter Firefighter 1 station code:
Invalid station code. Cannot be empty.
Program terminated.

```

Run E: invalid years active

```
Enter Firefighter 1 name      : Ahmad Zikri
Enter Firefighter 1 IC number : 900215105432
Enter Firefighter 1 phone     : 0123456789
Enter Firefighter 1 station code: FF-KL01
Enter Firefighter 1 years active: -3
Invalid years active. Must be between 0 and 40.
Program terminated.
```

Run F: invalid license number

```
Enter Medic 1 name           : Dr. Priya Devi
Enter Medic 1 IC number      : 920408025678
Enter Medic 1 phone          : 0112233445
Enter Medic 1 license number : 
Invalid license number. Cannot be empty.
Program terminated.
```

Run G: invalid specialisation

```
Enter Medic 1 name           : Dr. Priya Devi
Enter Medic 1 IC number      : 920408025678
Enter Medic 1 phone          : 0112233445
Enter Medic 1 license number : ML-2021-0042
Enter Medic 1 specialisation : 
Invalid specialisation. Cannot be empty.
Program terminated.
```

Run H: invalid badge ID

```
Enter Paramedic-FF badge ID : 
Invalid badge ID. Cannot be empty.
Program terminated.
```

Run I: invalid deployment status

```
Deployed? (1=Yes, 0=No)      : 2
Invalid deployment status. Must be 1 or 0.
```

 Reflection Questions

Answer the following questions **honestly and in your own words**. Write your answers as comments at the **bottom of your** .cpp file, after your `main()` function. There is no minimum or maximum word count; what matters is that your answer reflects **your own genuine experience** during this exercise.

Question 1: Describe **one error or bug** that you encountered while writing your code. What was the error message or symptom? What did you do to identify the cause, and how did you fix it?

Question 2: In this exercise, `Person` is declared as a virtual base class. Explain in your own words why this was necessary. What would happen inside `ParamedicFirefighter` if you removed the `virtual` keyword from both `Firefighter` and `Medic`? Give a specific reason based on your own code, not a general definition.

Question 3: What was the **most challenging part** of this exercise for your pair? How did you and your partner work through it together?

```
// =====
// REFLECTION
// =====
// Q1 - Error encountered:
//
//
//
```

```
// Q2 - Why Person is a virtual base class, and what happens without it:
//
//
//
// Q3 - Most challenging part:
//
//
//
// =====
```

⚠️ Reflection answers that are vague, copied from another pair, or clearly written by AI (e.g. no specific error mentioned, generic definitions, no personal experience) will receive **zero marks for the pair programming component**. Write what actually happened during your session.

Assessment

This exercise carries **3%** of the final course grade. The breakdown (out of 100 points) is as follows:

Criteria	Points
Problem Solving (Code)	
a. Task 1: Class hierarchy setup, virtual base class and inheritance structure for all four classes	15
b. Task 2: Mutator functions with correct input validation for all four classes	15
c. Task 3: Accessor functions and derived calculation method	10
d. Task 4: Redefined displayInfo() in all classes + correct explicit base class calls, no virtual/override	20
e. Task 5: main() + experience level report, formatted output	20
Pair Programming	
a. Active collaboration: Both members discuss, explain, and solve problems together throughout. Evidenced by video; the Driver explains the code out loud and the Navigator responds actively.	10
b. Role switching: Both members take turns as Driver and Navigator at least once. Evidenced by video (verbal announcement of role switch) and the Role Declaration comment in the source code.	10
Total	100

How pair programming is assessed: The lecturer will review the video recording and the Role Declaration comment in the source code. Selected pairs may be called for a short spot-check to verify understanding of their own code. Both the video and the spot-check (if applicable) contribute to the pair programming marks.

Submission

- Only one member from each pair needs to submit.
- Submit via **elearning** only. Submissions via email, Telegram, Google Drive, or any other platform will **not be accepted**.
- You must submit **THREE items**:
 - The source code: submit only the `.cpp` file. The file must contain the **Role Declaration comment** at the top and the **Reflection answers** as comments at the bottom.
 - The video link: submit the Google Drive link via **elearning**.
 - Note: The Reflection answers are embedded inside the `.cpp` file, not submitted separately.
- Deadline: **Sunday, 31st May 2026, 23:59 MYT (UTC+8)**.

1. **Who will be my partner?**
Your partner from Exercises 1 and 2.
2. **Can I pair up with someone from a different section?**
No. You must work with a partner from the same section.
3. **Can I do the exercise alone?**
Only if the number of students in the class is odd and you have obtained prior permission from the lecturer.
4. **Do we need to switch roles?**
Yes. Both members must experience both the Driver and Navigator roles at least once. The switch must be announced clearly in the video.
5. **Can I use AI tools such as ChatGPT or GitHub Copilot?**
You may use AI tools as a reference; for example, to understand a concept or look up syntax, the same way you would use a textbook. However, you must not use AI to generate your code for you. Your video must show that you understand and can explain every part of your code. If your explanation in the video is inconsistent with your submitted code, or if you cannot explain your code during a spot-check, you will receive zero marks for the pair programming component.
6. **What exactly must I say in the video when coding?**
While you are typing code, explain what you are doing and why. For example: "I am declaring Firefighter with virtual public Person because both Firefighter and Medic inherit from Person; without virtual, ParamedicFirefighter would contain two separate copies of Person." You do not need to read every line of code, but you must explain your design decisions and the OOP concepts you are applying.
7. **What is a spot-check?**
After submission, the lecturer may select certain pairs for a short verbal session (approximately 5 to 10 minutes) where each member is asked to explain parts of their submitted code. This is to confirm that the work is genuinely understood. You will be informed by the lecturer if your pair is selected.
8. **What if we encounter an error we cannot fix?**
Document it in your Reflection (Question 1). A program that does not fully work but includes an honest reflection about the error will be assessed more favourably than a suspiciously perfect submission with no reflection.
9. **What if my partner is not contributing?**
Inform the lecturer as soon as possible, before the deadline. Do not wait until after submission.
10. **What if we do pair programming face-to-face?**
You may sit side by side and use one computer. You do not need to use Live Share or an online meeting tool. Record the session locally using OBS or any screen recorder. Both members must still speak, discuss, and switch roles visibly in the video.
11. **Will code marks and pair programming marks be assessed separately?**
Yes. Code accounts for 80 points and pair programming accounts for 20 points. Both contribute to your final 100-point score.